



Project Report

Intern Name: Ahmed Islam

Department: Artificial Intelligence

Organization: IR Solutions

Designation: AI Intern

Week # 07

Submitted to: Asad Ali

Submission Date: 22-Aug-2025

Table of Contents

Project Report	1
Theoretical Understanding.....	4
Clustering	4
Type of Clustering:	4
Density based clustering	4
Distribution based clustering	4
Centroid Based Clustering	4
K-Means Clustering.....	5
How k-means clustering works?	5
GAN (Generative Adversarial Networks)	5
Generator Model	5
Generator Architecture	6
Generator loss	7
Discriminator Model	7
Discriminator Loss function	7
Project Work	8
Task 01: Random Forest Project	8
Description	8
Methodology	8
1. Data Exploration	8
2. Exploratory Data Analysis (EDA).....	8
3. Data Preprocessing	8
4. Handling Class Imbalance	8
5. Model Training.....	9
6. Model Evaluation	9
7. Model Saving.....	9
Task 02: Recommendation System Project.....	10
Description	10

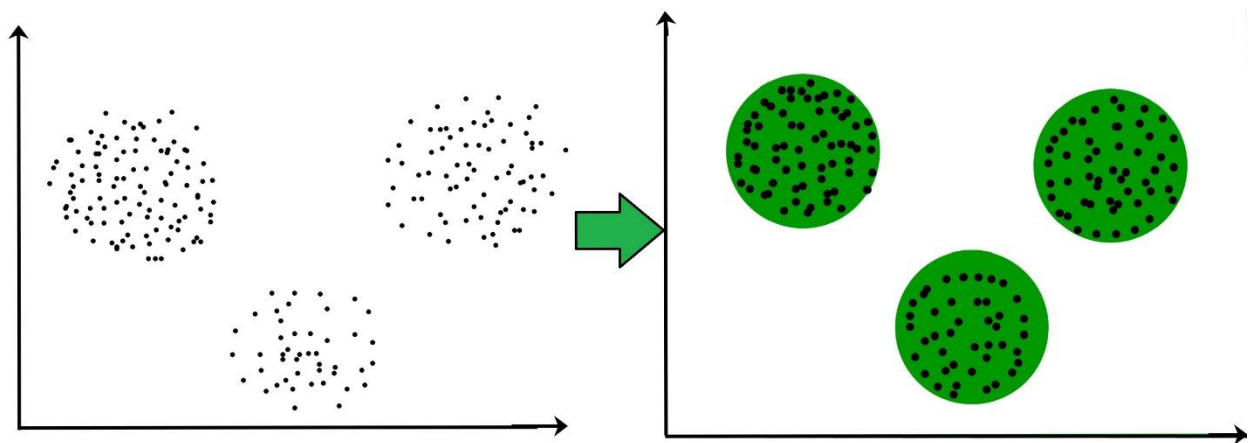
Methodology	10
1. Data Exploration	10
2. Exploratory Data Analysis (EDA)	10
3. Data Preprocessing	10
4. Model Building	11
5. Recommendation Function	11
6. Model Saving.....	11
Results	11
Example Recommendations	11
Challenges and Approaches	12
Results and Analysis	13
Decision Tree model trained on Imbalanced data.....	13
Random forest model trained on Imbalanced dataset	14
Smote Model Confusion Matrix.....	14
Feature Importance	15
Task 2 Result	15
Conclusion.....	15
References	16

Theoretical Understanding

Clustering

Clustering is an unsupervised machine learning technique designed to group unlabeled examples based on their similarity to each other.

After clustering, each group is assigned a unique label called a cluster ID. Clustering is powerful because it can simplify large, complex datasets with many features to a single cluster ID.



Type of Clustering:

Density based clustering

Density-based clustering connects contiguous areas of high example density into clusters. This allows for the discovery of any number of clusters of any shape. Outliers are not assigned to clusters.

Distribution based clustering

This approach assumes that the data is generated from a mixture of probability distributions (for example, Gaussian distributions).

The algorithm tries to find the parameters of these distributions and assign each data point to the distribution (cluster) it most likely belongs to.

Centroid Based Clustering

The centroid of a cluster is the arithmetic mean of all the points in the cluster. Centroid-based clustering organizes the data into non-hierarchical clusters.

K-Means Clustering

K-Means Clustering is an Unsupervised Machine Learning algorithm which groups unlabeled dataset into different clusters. It is used to organize data into groups based on their similarity.

How k-means clustering works?

We are given a data set of items with certain features and values for these features like a vector. The task is to categorize those items into groups. To achieve this we will use the K-means algorithm. 'K' in the name of the algorithm represents the number of groups/clusters we want to classify our items into.

- First we randomly initialize k points called means or cluster centroids.
- We categorize each item to its closest mean and we update the mean's coordinates, which are the averages of the items categorized in that cluster so far.
- We repeat the process for a given number of iterations and at the end, we have our clusters.

Selecting the right number of clusters is important for meaningful segmentation to do this we use Elbow Method for optimal value of k in KMeans which is a graphical tool used to determine the optimal number of clusters (k) in K-means.

GAN (Generative Adversarial Networks)

The main focus for GAN (Generative Adversarial Networks) is to generate data from scratch, mostly images but other domains including music have been done.

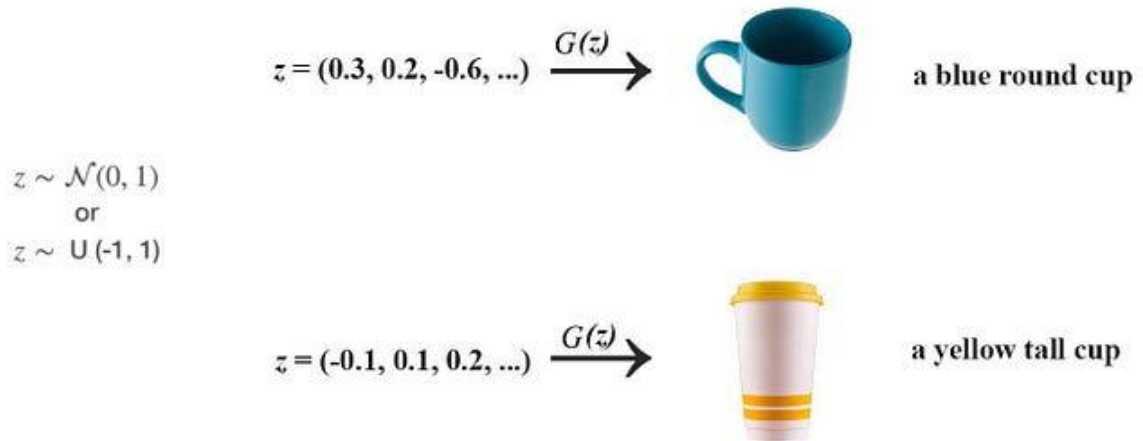
GAN composes of two deep networks, the generator, and the discriminator.

Generator Model

The generator is a deep neural network that takes random noise as input to generate realistic data samples like images or text. It learns the underlying data patterns by adjusting its internal parameters during training through backpropagation. Its objective is to produce samples that the discriminator classifies as real.

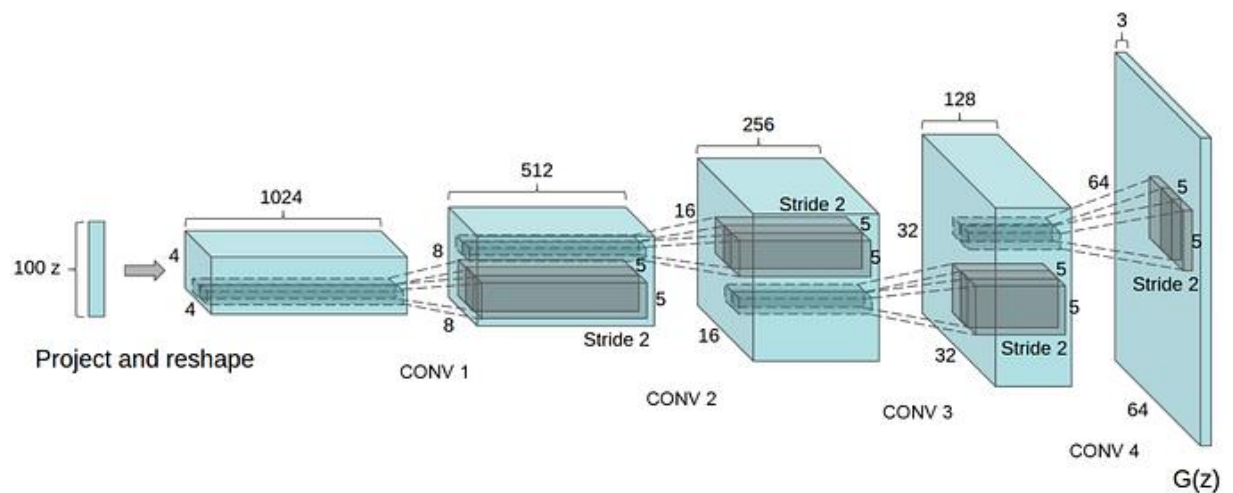
We will first look into how a generator creates images before learning how to train it.

First, we sample some noise z using a normal or uniform distribution. With z as an input, we use a generator G to create an image x ($x=G(z)$).



Conceptually, z represents the latent features of the images generated

Generator Architecture



Generator loss

Generator Loss Function: The generator tries to minimize this loss:

$$J_G = -\frac{1}{m} \sum_{i=1}^m \log D(G(z_i))$$

where

- J_G measure how well the generator is fooling the discriminator.
- $G(z_i)$ is the generated sample from random noise z_i
- $D(G(z_i))$ is the discriminator's estimated probability that the generated sample is real.

The generator aims to maximize $D(G(z_i))$ meaning it wants the discriminator to classify its fake data as real (probability close to 1).

Discriminator Model

The discriminator acts as a binary classifier helps in distinguishing between real and generated data. It learns to improve its classification ability through training, refining its parameters to detect fake samples more accurately.

Discriminator Loss function

Discriminator Loss Function: The discriminator tries to minimize this loss:

$$J_D = -\frac{1}{m} \sum_{i=1}^m \log D(x_i) - \frac{1}{m} \sum_{i=1}^m \log(1 - D(G(z_i)))$$

- J_D measures how well the discriminator classifies real and fake samples.
- x_i is a real data sample.
- $G(z_i)$ is a fake sample from the generator.
- $D(x_i)$ is the discriminator's probability that x_i is real.
- $D(G(z_i))$ is the discriminator's probability that the fake sample is real.

The discriminator wants to correctly classify real data as real (maximize $\log D(x_i)$) and fake data as fake (maximize $\log(1 - D(G(z_i)))$)

Project Work

Task 01: Random Forest Project

Description

The objective of this task was to predict whether a borrower will fully repay a loan using the LendingClub dataset (2007–2010). Both Decision Tree and Random Forest models were implemented and compared. Since the dataset was imbalanced, multiple resampling strategies were applied to improve model performance.

Methodology

1. Data Exploration

- Loaded dataset (loan_data.csv) using **pandas**.
- Initial inspection with `info()`, `describe()`, and `head()` revealed categorical (purpose) and numerical features, along with imbalance in the target column `not.fully.paid`.

2. Exploratory Data Analysis (EDA)

Visualizations were performed to understand feature distributions and relationships:

- **Histogram** of FICO scores by `credit.policy`.
- **Histogram** of FICO scores by `not.fully.paid`.
- **Countplot** of loans by purpose, colored by `not.fully.paid`.
- **Jointplot** of FICO score vs. interest rate.
- **Implot** of interest rate vs. FICO score, separated by `not.fully.paid` and `credit.policy`.

3. Data Preprocessing

- Converted categorical purpose column into dummy variables using `pd.get_dummies()`.
- Split dataset into **70% training** and **30% testing** sets using `train_test_split(random_state=101)`.

4. Handling Class Imbalance

Since the dataset was **imbalanced** (more borrowers fully repaid loans compared to defaults), four balancing techniques were applied:

- **Undersampling:** Reduced the majority class to match minority samples.
- **Oversampling:** Replicated minority samples to balance with the majority.
- **SMOTE (Synthetic Minority Over-sampling Technique):** Generated synthetic samples for the minority class.
- **Class Weighting:** Used scikit-learn's `class_weight="balanced"` option to penalize misclassification of the minority class.

5. Model Training

- **Decision Tree Classifier:** Baseline model trained using scikit-learn's `DecisionTreeClassifier`.
- **Random Forest Classifier:** Trained using `RandomForestClassifier(n_estimators=600)` for robust performance.

6. Model Evaluation

Both models were evaluated using:

- **Accuracy**
- **Precision**
- **Recall**
- **F1-score**
- **Confusion Matrix**

Visualizations included:

- Confusion matrices (heatmaps).
- Feature importance plots (Random Forest).
- Distribution plots comparing balanced vs. imbalanced datasets.

7. Model Saving

The **best-performing model** (Random Forest with SMOTE + class weighting) was saved using **Pickle** for future deployment.

Task 02: Recommendation System Project

Description

The objective of this task was to build a book recommendation system using the Goodreads dataset (books.csv). A k-Nearest Neighbors (k-NN) algorithm was applied to suggest books similar to a given input based on metadata such as ratings, number of ratings, authors, publishers, and language.

Methodology

1. Data Exploration

- Loaded the dataset using **pandas** and inspected its structure using `info()`, `describe()`, and `head()`.
- Dataset included features such as:
 - `book_id`, `title`, `authors`, `average_rating`, `ratings_count`, `language_code`, `num_pages`, `publisher`, and `text_reviews_count`.

2. Exploratory Data Analysis (EDA)

Visualizations provided insights into the dataset distribution and characteristics:

- **Bar Plot:** Top 10 books with over 1M ratings, sorted by average rating.
- **Bar Plot:** Top 10 authors by number of books.
- **Bar Plot:** Top 10 most-rated books by ratings count.
- **Countplot:** Books by `language_code` (top 10 languages).
- **Bar Plot:** Top 10 highly rated authors (books with average rating > 4.4).
- **Bar Plot:** Top 10 publishers by number of books.
- **Distribution Plot:** Distribution of average ratings.
- **Relplots:** Average rating vs. ratings count, number of pages, `language_code`, and text reviews count.

3. Data Preprocessing

- **Feature Engineering:**

- Created a rating_between column by binning average_rating into categories (0–1, 1–2, 2–3, 3–4, 4–5).
- Converted rating_between and language_code into **dummy variables**.
- **Feature Matrix Construction:**
 - Combined dummy variables with average_rating and ratings_count.
- **Feature Scaling:**
 - Applied **MinMaxScaler** to normalize the feature matrix, ensuring fair distance computation in k-NN.

4. Model Building

- Implemented a **k-Nearest Neighbors (k-NN)** model with:
 - n_neighbors=6
 - algorithm='ball_tree' (efficient for large datasets with sparse features).
- Fitted the model on the **scaled feature matrix**.

5. Recommendation Function

- Created a BookRecommender function:
 - Input: Book title.
 - Output: List of **recommended book titles** based on nearest neighbors in feature space.
- Tested with **two sample book titles** to evaluate recommendation quality.

6. Model Saving

- The trained k-NN model was serialized using **Pickle** for future use and deployment.

Results

Example Recommendations

Input Book 1: *The Hunger Game*

- Recommended Books:

	title	authors	average_rating	ratings_count	publisher	distance
0	Bite Club (Bite Club #1)	Howard Chaykin/David Tischman/David Hahn	3.07	148	Vertigo (DC Comics)	0.000007
1	The Devil in the Junior League	Linda Francis Lee	3.62	3852	St. Martin's Press	0.000012
2	Deep Waters (Cetin Ikmen #4)	Barbara Nadel	3.99	301	Headline	0.000066
3	Heart of Darkness	Joseph Conrad	3.42	92	Wildside Press	0.000068
4	The Science of Stephen King: From 'Carrie' to ...	Lois H. Gresh/Robert E. Weinberg	3.42	62	Wiley	0.000075

Input Book 2: *Twilight*

- Recommended Books:

	title	authors	average_rating	ratings_count	publisher	distance
0	Plain Truth	Jodi Picoult	3.98	143658	Atria Books	0.001209
1	The Wind in the Willows	Kenneth Grahame/Gillian Avery	3.99	147748	Penguin Books	0.002899
2	The Wedding (The Notebook #2)	Nicholas Sparks	3.99	126629	Vision	0.003198
3	Heidi (Heidi #1-2)	Johanna Spyri/Beverly Cleary/Angelo Rinaldi	3.99	153317	Kingfisher	0.003867
4	Sorrow's Anthem (Lincoln Perry #2)	Michael Koryta	3.89	62	St. Martin's Paperbacks	0.004026

Challenges and Approaches

Challenge 1: Imbalanced Dataset

- Description:**

The target variable not.fully.paid was highly imbalanced, with significantly fewer borrowers defaulting compared to those who fully repaid.

- This imbalance caused the models (Decision Tree and Random Forest) to be biased toward the majority class, leading to high accuracy but poor **recall** for the minority (default) class.

- Approaches Applied:**

1. Undersampling:

- Reduced the majority class size to balance with the minority class.
- Helped improve recall but caused loss of valuable data, reducing overall accuracy.

2. Oversampling:

- Replicated minority class samples to balance the dataset.

- Preserved all majority class data but risked overfitting due to duplicate records.

3. **SMOTE (Synthetic Minority Over-sampling Technique):**

- Generated synthetic minority samples by interpolating between existing records.
- Achieved a better trade-off between precision and recall without significant overfitting.

4. **Class Weight Assignment:**

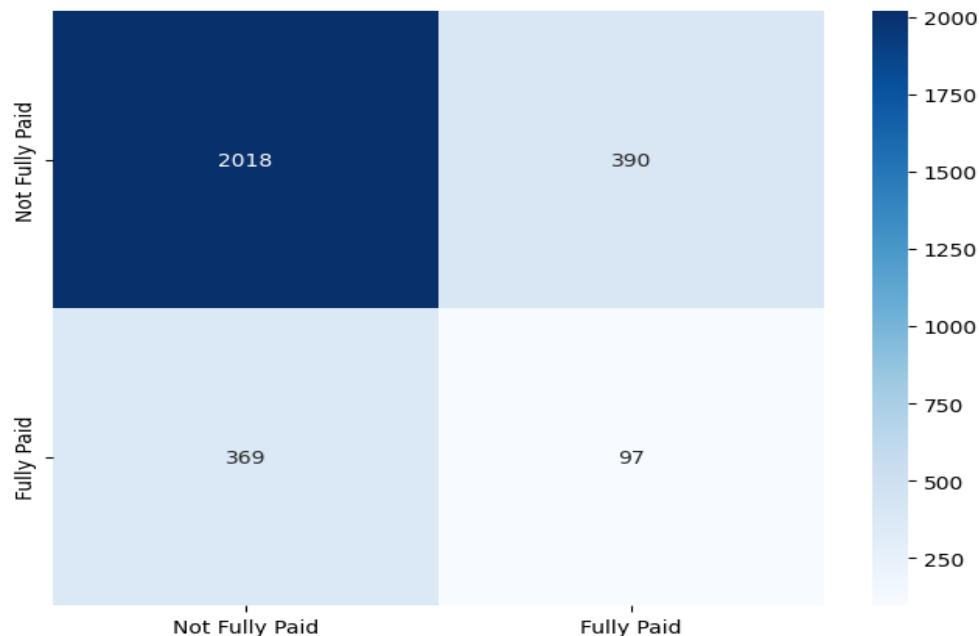
- Applied `class_weight="balanced"` in scikit-learn classifiers.
- Automatically penalized misclassification of the minority class, improving minority detection without changing the dataset.

- **Outcome:**

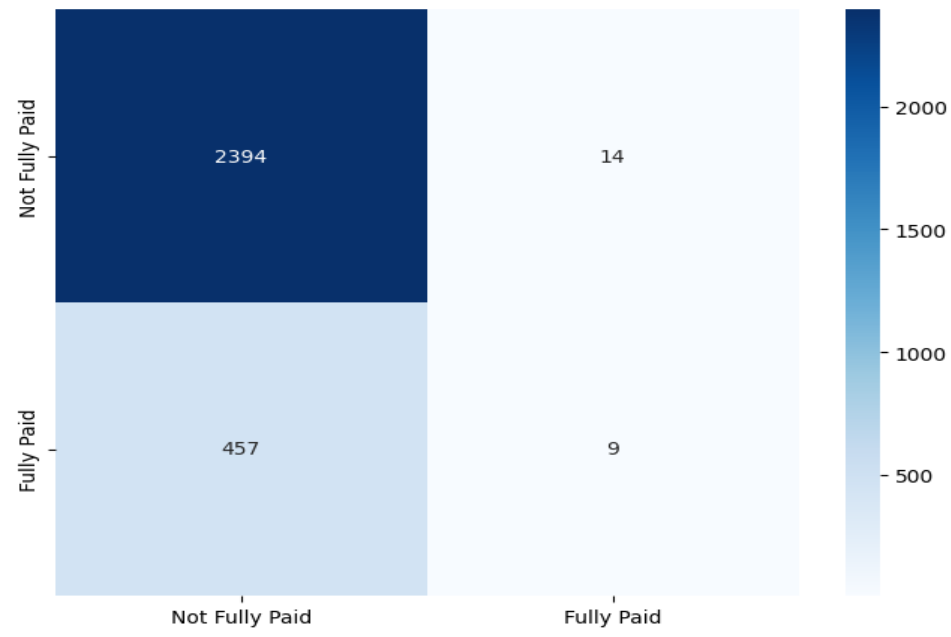
Among the techniques, **SMOTE combined with Random Forest** gave the most balanced performance, achieving higher recall for default prediction while maintaining competitive precision and F1-score.

Results and Analysis

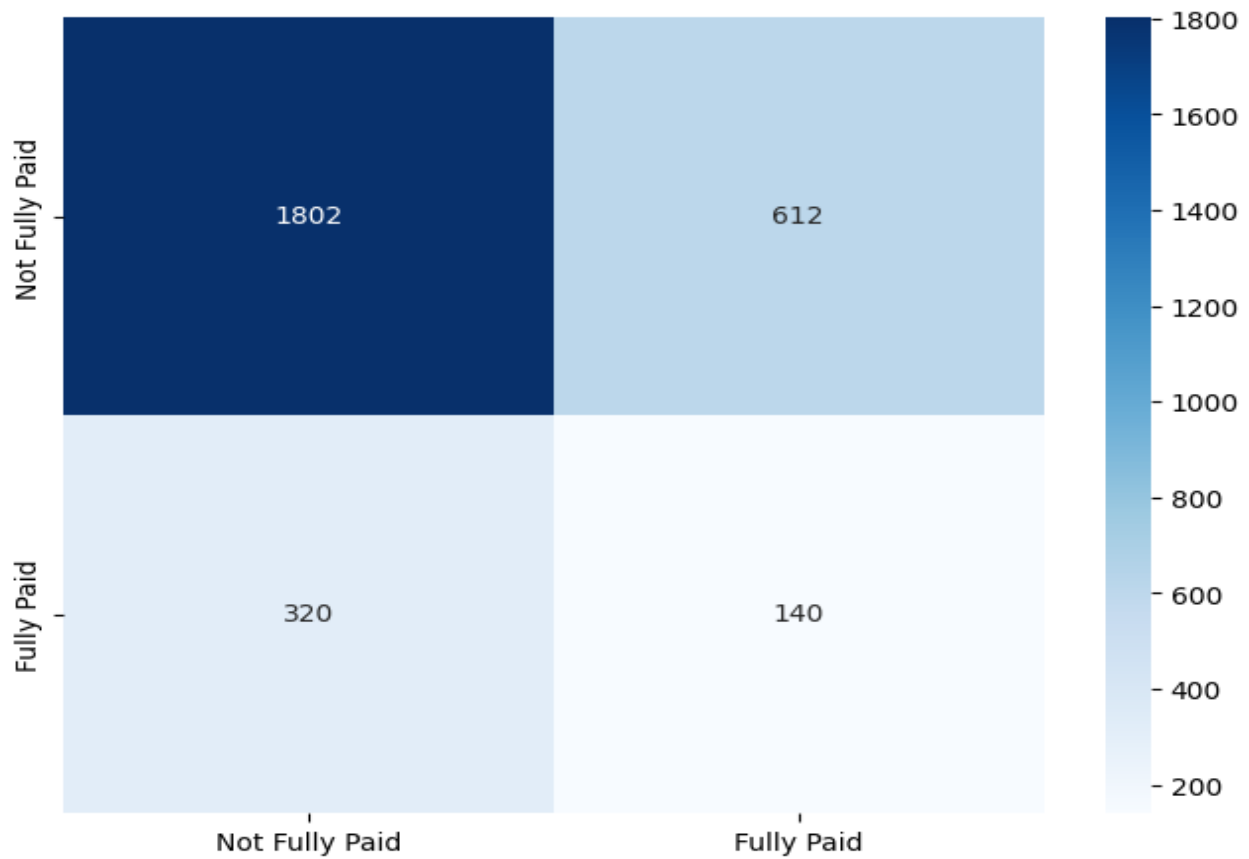
Decision Tree model trained on Imbalanced data



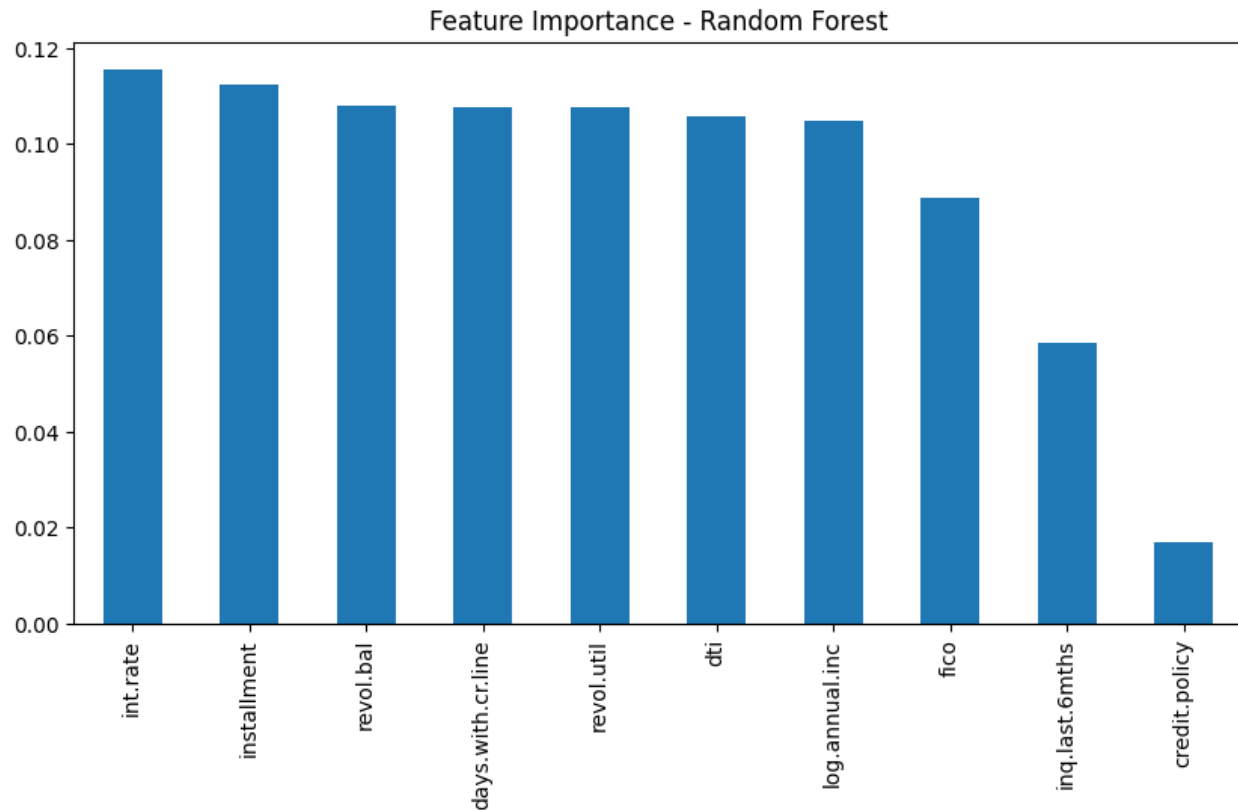
Random forest model trained on Imbalanced dataset



Smote Model Confusion Matrix



Feature Importance



Task 2 Result

```
try:
    display(BookRecommender("The Hunger Games", n_recs=5).head())
except Exception as e:
    print("Test with 'The Hunger Games':", e)
```

	title	authors	average_rating	ratings_count	publisher	distance
0	Bite Club (Bite Club #1)	Howard Chaykin/David Tischman/David Hahn	3.07	148	Vertigo (DC Comics)	0.000007
1	The Devil in the Junior League	Linda Francis Lee	3.62	3852	St. Martin's Press	0.000012
2	Deep Waters (Cetin Ikmen #4)	Barbara Nadel	3.99	301	Headline	0.000066
3	Heart of Darkness	Joseph Conrad	3.42	92	Wildside Press	0.000068
4	The Science of Stephen King: From 'Carrie' to ...	Lois H. Gresh/Robert E. Weinberg	3.42	62	Wiley	0.000075

Conclusion

This week, I deepened my understanding of K-Means clustering and GANs, alongside implementing two practical projects. In the Random Forest project, I tackled class imbalance using undersampling, oversampling, SMOTE, and class weighting, achieving improved results with Random Forest + SMOTE. In the Recommendation System project, I built a k-NN-based model that generated meaningful book suggestions, highlighting the

importance of feature engineering and scaling. Overall, the week enhanced both my theoretical knowledge and practical machine learning skills.

References

<https://www.geeksforgeeks.org/machine-learning/k-means-clustering-introduction/>

https://en.wikipedia.org/wiki/Generative_adversarial_network

<https://developers.google.com/machine-learning/gan>